

Regression Exercise - Documented Solution

```
In [1]: from regression_init import LOGGER, RANDOM_SEED, RESOURCE_DIRECTORY

LOGGER.info(
    msg=f"Regression exercise initialized with seed {RANDOM_SEED} "
    f"and resources at {RESOURCE_DIRECTORY}"
)
```

2026-07-27 15:41:52,986 | INFO | regression_exercise | Regression exercise initialized with seed 20260727 and resources at C:\Users\asherk\PycharmProjects\DS_Training\D - ML\regression\exercise\resources

In this exercise we explore linear and logistic regression, polynomial basis expansion, regularization, feature engineering, and KNN regression. Every stochastic operation uses a fixed seed, and model parameters and hyperparameters are learned with training data or training-only cross-validation. Test target values are used only for final metrics. To follow the dataframe-level feature-engineering instructions exactly, target-free category vocabularies, ranks, and same-year reference means are computed on the fixed feature table before the seeded split. This is transductive--it uses held-out covariates but never held-out SalePrice values--so a deployment pipeline should fit those preprocessors on training rows only.

```
In [2]: import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sklearn
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from regression_utils import (
    DEFAULT_COEFFICIENTS,
    add_bonus_house_feature,
    add_one_hot_house_features,
    add_required_house_features,
    build_polynomial_features,
    evaluate_regressor,
    fit_knn_search,
    fit_ridge_search,
    generate_linear_regression_data,
    generate_logistic_regression_data,
    select_numeric_house_data,
    split_regression_data,
)

plt.style.use(style="seaborn-v0_8-whitegrid")
pd.options.display.float_format = "{:,.4f}".format

environment_summary = pd.DataFrame(
```

```

data=[
    {"component": "Python", "version": sys.version.split()[0]},
    {"component": "NumPy", "version": np.__version__},
    {"component": "pandas", "version": pd.__version__},
    {"component": "scikit-learn", "version": sklearn.__version__},
]
).set_index(keys="component")
environment_summary

```

Out[2]:

	version
component	
Python	3.14.6
NumPy	2.5.1
pandas	3.0.3
scikit-learn	1.9.0

Coefficients and some more

We will start by exploring several concepts regarding linear and logistic regression. To that matter, we will prefer working using a generated dataset. Start by generating (using numpy and np.random) the following dataset:

$X \sim \mathcal{N}(0, 1)^3$ (i.e X consists of 3-dimensional vectors)

$Y = 0.3 \cdot X[:, 0] + 0.5 \cdot X[:, 1] - 0.7 \cdot X[:, 2] + 1$

Generate 1,000 samples.

Solution: generate the linear data

Three approaches considered

1. Use the legacy global `np.random.normal` state.
2. Use `sklearn.datasets.make_regression`, then retrofit the exact requested equation.
3. Use NumPy's independent `default_rng` and vectorized matrix multiplication.

Choice: approach 3 is the most direct and reproducible. It represents the equation as $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + 1$ with a fixed seed, introduces no unintended noise, and avoids mutable global random state. The typed helper validates its inputs.

Direct answer: the generated feature matrix has shape `(1000, 3)`: every row is one three-dimensional observation and every column is sampled independently from $\mathcal{N}(0, 1)$. The target is a length-1,000 vector computed without added noise. Its observed mean is `0.9917`, close to the theoretical mean of `1`, and its observed standard deviation is `0.9072`, close to $\sqrt{0.3^2 + 0.5^2 + (-0.7)^2} = \sqrt{0.83} \approx 0.911$. The fixed seed makes the exercise repeatable; it does not change the requested distribution.

```
In [3]: linear_features, linear_target = generate_linear_regression_data(
        number_of_samples=1_000,
        random_seed=RANDOM_SEED,
    )

linear_generated_data = pd.DataFrame(
    data=linear_features,
    columns=["x_0", "x_1", "x_2"],
).assign(y=linear_target)

pd.DataFrame(
    data=[
        {
            "samples": linear_features.shape[0],
            "features": linear_features.shape[1],
            "target_mean": linear_target.mean(),
            "target_standard_deviation": linear_target.std(),
        }
    ]
)
```

```
Out[3]:
```

	samples	features	target_mean	target_standard_deviation
0	1000	3	0.9917	0.9072

Train a simple linear regression

(`sklearn.linear_model.LinearRegression`) on the data. What coefficients did you get using your regression? Did you expect those coefficients?

Solution: recover the linear coefficients

Three approaches considered

1. Solve the normal equations with NumPy.
2. Use `scipy.linalg.lstsq`, which is numerically safer than explicitly inverting $X^T X$.
3. Use sklearn's ready-made `LinearRegression`.

Choice: approach 3 matches the exercise API and internally solves least squares robustly. Because the data are noiseless and the design is full rank, the expected coefficients are exactly $(0.3, 0.5, -0.7)$ and the intercept is 1, up to floating-point round-off.

Direct answer: sklearn recovered coefficients `[0.3000, 0.5000, -0.7000]` and an intercept of `1.0000`. Yes, these values are exactly what we should expect. The target was constructed from that same linear equation, no random error was added, and 1,000 rows are more than enough for the three feature columns to have full rank. Therefore the unique least-squares solution has zero residual error and matches the generating parameters; the displayed absolute errors are zero at four decimal places.

```
In [4]: linear_regression_model = LinearRegression()
linear_regression_model.fit(
    X=linear_features,
```

```

    y=linear_target,
)

linear_coefficient_results = pd.DataFrame(
    data={
        "parameter": ["intercept", "x_0", "x_1", "x_2"],
        "expected": [1.0, *DEFAULT_COEFFICIENTS.tolist()],
        "estimated": [
            linear_regression_model.intercept_,
            *linear_regression_model.coef_.tolist(),
        ],
    }
).set_index(keys="parameter")
linear_coefficient_results.loc[:, "absolute_error"] = (
    linear_coefficient_results.loc[:, "estimated"]
    - linear_coefficient_results.loc[:, "expected"]
).abs()
linear_coefficient_results

```

Out[4]:

	expected	estimated	absolute_error
parameter			

parameter	expected	estimated	absolute_error
intercept	1.0000	1.0000	0.0000
x_0	0.3000	0.3000	0.0000
x_1	0.5000	0.5000	0.0000
x_2	-0.7000	-0.7000	0.0000

We will now conduct a similar experiment, only this time using logistic regression, with minor adjustments:

$X \sim \mathcal{N}(0, 1)^3$ (i.e X consists of 3-dimensional vectors)

$Y = (0.3 \cdot X[:, 0] + 0.5 \cdot X[:, 1] - 0.7 \cdot X[:, 2] \geq 1)$

Generate 1,000 samples.

Solution: generate deterministic binary labels

Three approaches considered

1. Label rows in a Python loop.
2. Generate a generic classification dataset with `make_classification`.
3. Compute the requested score with NumPy and apply the threshold in one vectorized expression.

Choice: approach 3 exactly preserves the stated decision boundary and is both simpler and faster than a row loop. A generic generator would solve a different problem.

Direct answer: the Boolean comparison creates 1,000 labels using the boundary $0.3x_0 + 0.5x_1 - 0.7x_2 - 1 = 0$. A row is class `1` on or above that plane and class `0` otherwise. This seeded sample contains `142` positive and `858` negative examples. The

imbalance is expected: the score has standard deviation about `0.911`, while the positive threshold is `1`, so only observations in the upper tail become class `1`.

```
In [5]: logistic_features_1k, logistic_target_1k = (
        generate_logistic_regression_data(
            number_of_samples=1_000,
            random_seed=RANDOM_SEED,
            decision_threshold=1.0,
        )
    )
    logistic_class_counts = pd.Series(
        data=logistic_target_1k,
        name="class",
    ).value_counts(sort=False)
    pd.DataFrame(
        data={
            "class": logistic_class_counts.index,
            "count": logistic_class_counts.to_numpy(),
        }
    ).set_index(keys="class")
```

Out[5]:

	count
class	
0	858
1	142

Train a simple logistic regression (`sklearn.linear_model.LogisticRegression`) on the data. What coefficients did you get using your logistic regression? Did you expect those coefficients? Why, or why not?

Solution: fit and interpret logistic regression

Three approaches considered

1. Optimize the log-loss manually with `scipy.optimize`.
2. Fit a binomial generalized linear model.
3. Use sklearn's regularized `LogisticRegression`.

Choice: approach 3 is the ready-made, stable estimator requested. The raw coefficients should **not** equal $(0.3, 0.5, -0.7)$: the labels are perfectly separable, and multiplying the entire boundary (β, b) by any positive constant leaves every class unchanged. Without regularization, finite maximum-likelihood coefficients do not exist. sklearn's default L2 penalty chooses one finite scale, so direction and boundary accuracy are the meaningful comparisons.

Direct answer: the fitted coefficients are approximately `[2.1076, 3.8739, -5.3825]`, with intercept `-7.7507`, and training accuracy is `0.993`. These raw numbers are not expected to equal `[0.3, 0.5, -0.7]`. Logistic regression estimates log-odds, whereas the data contain hard threshold labels rather than probabilities

generated by a logistic curve. In addition, the classes are separable, so the boundary's scale is not identifiable. After dividing out the fitted positive scale, the estimated boundary is approximately $[0.2738, 0.5032, -0.6992, -1.0068]$, which is very close to the true $[0.3, 0.5, -0.7, -1]$. Thus the orientation and location are recovered even though the unscaled coefficients differ.

```
In [6]: logistic_regression_model_1k = LogisticRegression(  
    C=1.0,  
    penalty="l2",  
    solver="lbfgs",  
    max_iter=20_000,  
    tol=1e-10,  
    random_state=RANDOM_SEED,  
)  
logistic_regression_model_1k.fit(  
    X=logistic_features_1k,  
    y=logistic_target_1k,  
)  
  
true_boundary = np.asarray(  
    a=[*DEFAULT_COEFFICIENTS.tolist(), -1.0],  
    dtype=np.float64,  
)  
estimated_boundary_1k = np.asarray(  
    a=[  
        *logistic_regression_model_1k.coef_[0].tolist(),  
        logistic_regression_model_1k.intercept_[0],  
    ],  
    dtype=np.float64,  
)  
estimated_positive_scale_1k = (  
    estimated_boundary_1k @ true_boundary  
)/ (true_boundary @ true_boundary)  
  
logistic_boundary_results = pd.DataFrame(  
    data={  
        "parameter": ["x_0", "x_1", "x_2", "intercept"],  
        "true_boundary": true_boundary,  
        "estimated": estimated_boundary_1k,  
        "estimated_rescaled": (  
            estimated_boundary_1k / estimated_positive_scale_1k  
        ),  
    }  
).set_index(keys="parameter")  
logistic_boundary_results.loc[:, "training_accuracy"] = (  
    logistic_regression_model_1k.score(  
        X=logistic_features_1k,  
        y=logistic_target_1k,  
    )  
)  
logistic_boundary_results
```

Out[6]:

	true_boundary	estimated	estimated_rescaled	training_accuracy
parameter				
x_0	0.3000	2.1076	0.2738	0.9930
x_1	0.5000	3.8739	0.5032	0.9930
x_2	-0.7000	-5.3825	-0.6992	0.9930
intercept	-1.0000	-7.7507	-1.0068	0.9930

Repeat this experiment, this time with 10,000 samples and 100,000 samples. Did you get different coefficients? Why or why not?

Solution: repeat at larger sample sizes

Three approaches considered

1. Refit the same deterministic sklearn model for each requested sample size.
2. Train one incremental `SGDClassifier` as samples arrive.
3. Analyze only the theoretical asymptotic boundary.

Choice: approach 1 answers the empirical question directly and keeps the estimator unchanged. The same seed makes the datasets nested. For separable data, coefficient norms typically grow as more observations support a sharper boundary relative to the fixed regularization, while normalized direction, cosine similarity, and accuracy stabilize.

Direct answer: yes, the coefficients change substantially. The boundary norm grows from 10.4161 at 1,000 samples, to 22.5692 at 10,000, and to 48.9964 at 100,000. For example, the first coefficient grows from 2.1076 to 4.9205 and then 10.8929. This does not mean the decision boundary is moving: its cosine similarity to the true boundary is already 0.9998 and rounds to 1.0000 for the larger samples, while training accuracy rises from 0.9930 to 0.9997. Because the data are separable and the L2 penalty is fixed, more evidence lets the optimizer choose a larger-magnitude, sharper logistic transition. Direction is therefore more informative than raw magnitude.

```
In [7]: logistic_experiment_rows: list[dict[str, float | int]] = []
true_boundary_norm = (true_boundary @ true_boundary) ** 0.5

for sample_count in (1_000, 10_000, 100_000):
    current_features, current_target = generate_logistic_regression_data(
        number_of_samples=sample_count,
        random_seed=RANDOM_SEED,
        decision_threshold=1.0,
    )
    current_model = LogisticRegression(
        C=1.0,
        penalty="l2",
        solver="lbfgs",
        max_iter=20_000,
        tol=1e-10,
        random_state=RANDOM_SEED,
    )
```

```

current_model.fit(
    X=current_features,
    y=current_target,
)
current_boundary = np.asarray(
    a=[
        *current_model.coef_[0].tolist(),
        current_model.intercept_[0],
    ],
    dtype=np.float64,
)
current_boundary_norm = (current_boundary @ current_boundary) ** 0.5
logistic_experiment_rows += [
    {
        "samples": sample_count,
        "coef_x0": current_model.coef_[0, 0],
        "coef_x1": current_model.coef_[0, 1],
        "coef_x2": current_model.coef_[0, 2],
        "intercept": current_model.intercept_[0],
        "boundary_norm": current_boundary_norm,
        "cosine_to_true_boundary": (
            (current_boundary @ true_boundary)
            / (current_boundary_norm * true_boundary_norm)
        ),
        "training_accuracy": current_model.score(
            X=current_features,
            y=current_target,
        ),
    }
]

logistic_sample_size_results = pd.DataFrame(
    data=logistic_experiment_rows,
).set_index(keys="samples")
logistic_sample_size_results.transpose()

```

Out[7]:

	samples	1000	10000	100000
coef_x0		2.1076	4.9205	10.8929
coef_x1		3.8739	8.2941	18.1022
coef_x2		-5.3825	-11.6298	-25.3105
intercept		-7.7507	-16.7665	-36.2447
boundary_norm		10.4161	22.5692	48.9964
cosine_to_true_boundary		0.9998	1.0000	1.0000
training_accuracy		0.9930	0.9971	0.9997

Non-linear linear regressions

Load

Can be found in: https://drive.google.com/open?id=1y3HtVk0N1q4xYn_qczDcdkZfGpy23z91``

Solution: load the nonlinear-function training data

Three approaches considered

1. Parse the CSV with Python's `csv` module.
2. Load numeric columns with `numpy.genfromtxt`.
3. Use `pandas.read_csv`.

Choice: approach 3 preserves column names, integrates cleanly with plotting and sklearn, and is the most readable ready-made loader. The real file linked by the exercise is stored in `resources/`.

Direct answer: `func_1_train.csv` loads successfully as a dataframe with 200 rows and the two numeric columns `x` and `y`. The observed `x` values span roughly 0.0049 to 0.9934, while `y` spans about -0.3004 to 0.0527. Keeping the names is useful because all later feature construction and plots can address the predictor and target explicitly.

```
In [8]: function_training_data = pd.read_csv(
        filepath_or_buffer=RESOURCE_DIRECTORY / "func_1_train.csv",
    )
function_training_data.describe()
```

```
Out[8]:
```

	x	y
count	200.0000	200.0000
mean	0.4800	-0.1677
std	0.2873	0.0789
min	0.0049	-0.3004
25%	0.2272	-0.2320
50%	0.4772	-0.1855
75%	0.7261	-0.1134
max	0.9934	0.0527

Draw a scatter plot of `y` as a function of `x`. What kind of functions would you like to fit here?

Solution: inspect the function shape

Three approaches considered

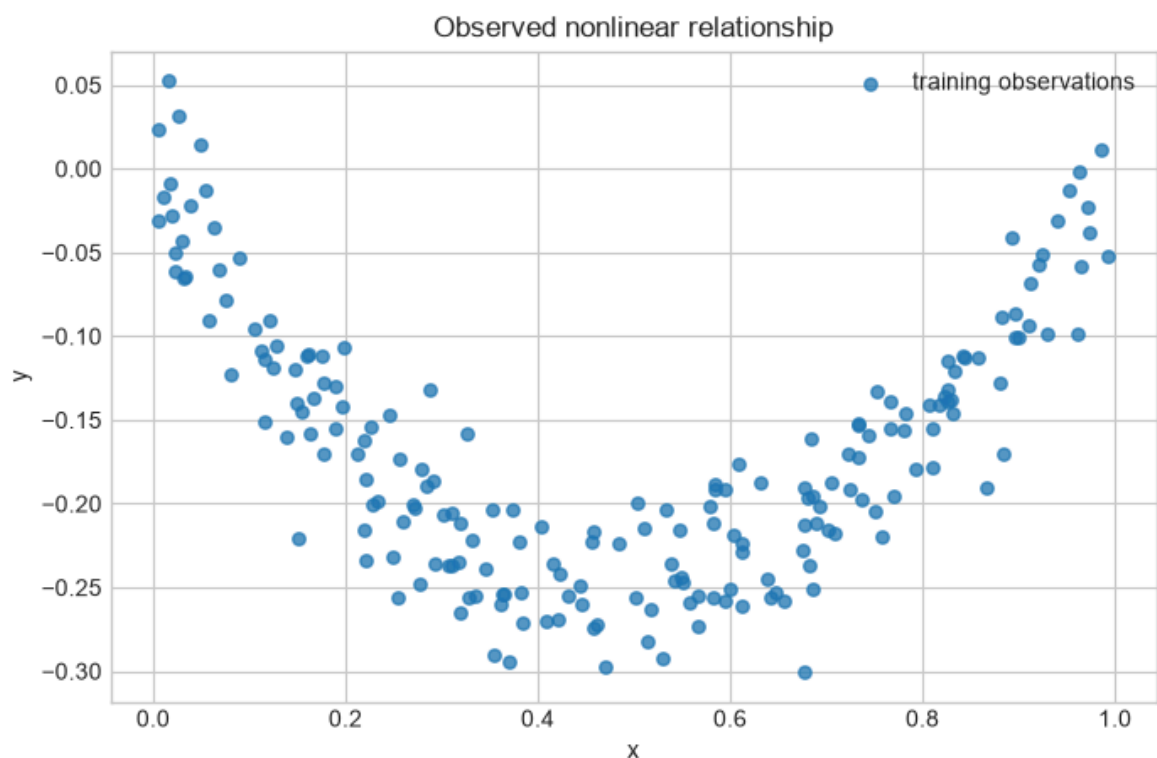
1. A polynomial basis for a globally smooth curve.
2. Cubic splines for a smooth piecewise fit with local control.
3. Fourier or kernel features for an explicitly oscillatory pattern.

Choice: first inspect the scatter plot, then use a polynomial basis because the next question explicitly requests powers through 49. Splines would often be the practical

choice, but they would not demonstrate the intended linear-in-the-parameters idea.

Direct answer: the scatter plot is a smooth, convex U-shaped relationship with its minimum near $x = 0.5$; it resembles a noisy parabola rather than a straight line. A quadratic polynomial is the simplest sensible candidate. Higher-order polynomials or cubic splines could accommodate small departures from an exact parabola. A periodic model is not suggested because the plot shows one bowl rather than repeated cycles.

```
In [9]: function_scatter_figure, function_scatter_axes = plt.subplots(
    nrows=1,
    ncols=1,
    figsize=(8, 5),
)
function_scatter_axes.scatter(
    x=function_training_data.loc[:, "x"],
    y=function_training_data.loc[:, "y"],
    s=28,
    alpha=0.75,
    label="training observations",
)
function_scatter_axes.set(
    title="Observed nonlinear relationship",
    xlabel="x",
    ylabel="y",
)
function_scatter_axes.legend()
plt.show()
```



Try fitting a linear regression. Draw the data points and the function you fitted on the same plot.

Solution: establish a straight-line baseline

Three approaches considered

1. `numpy.polyfit` with degree one.
2. `scipy.linalg.lstsq` on a manually augmented matrix.
3. sklearn `LinearRegression`.

Choice: approach 3 is consistent with the rest of the exercise and provides the required baseline. Its high bias should be visible because one straight line cannot represent the curved relationship.

Direct answer: the fitted red line misses the U-shape and runs close to the overall mean of `y`. Its training MSE is about `0.0062` and its training R^2 is only `0.0061`, so it explains less than one percent of the observed variation. This is underfitting: because low and high `x` values both have relatively high `y`, their opposing linear effects mostly cancel, leaving a nearly flat best straight line.

```
In [10]: function_training_features = function_training_data.loc[
        :,
        ["x"],
    ].to_numpy(dtype=np.float64)
    function_training_target = function_training_data.loc[:, "y"].to_numpy(
        dtype=np.float64,
    )

    linear_function_model = LinearRegression()
    linear_function_model.fit(
        X=function_training_features,
        y=function_training_target,
    )
    linear_function_prediction = linear_function_model.predict(
        X=function_training_features,
    )
    linear_function_train_metrics = evaluate_regressor(
        regression_model=linear_function_model,
        feature_data=function_training_features,
        target_data=function_training_target,
    )

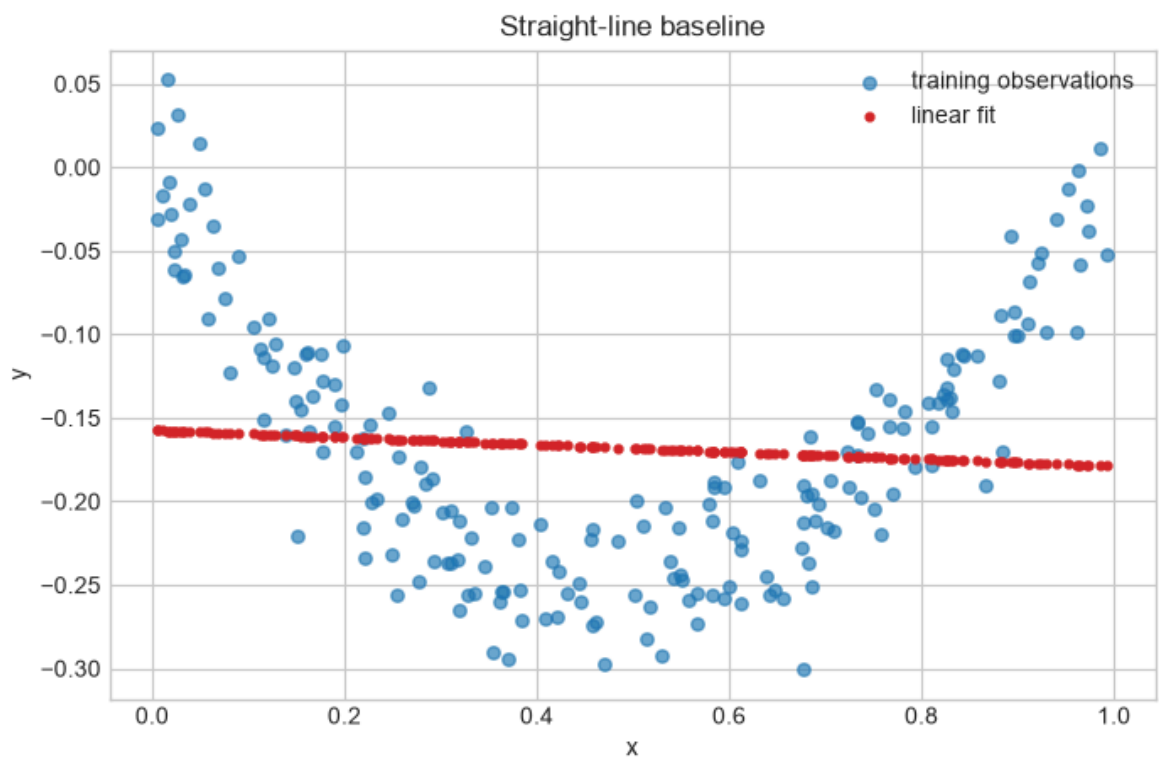
    linear_function_figure, linear_function_axes = plt.subplots(
        nrows=1,
        ncols=1,
        figsize=(8, 5),
    )
    linear_function_axes.scatter(
        x=function_training_data.loc[:, "x"],
        y=function_training_data.loc[:, "y"],
        s=28,
        alpha=0.65,
        label="training observations",
    )
    linear_function_axes.scatter(
        x=function_training_data.loc[:, "x"],
        y=linear_function_prediction,
```

```

s=12,
color="tab:red",
label="linear fit",
)
linear_function_axes.set(
    title="Straight-line baseline",
    xlabel="x",
    ylabel="y",
)
linear_function_axes.legend()
plt.show()

pd.DataFrame(
    data=[
        {
            "model": "LinearRegression(x)",
            "training_mse": linear_function_train_metrics.mean_squared_error,
            "training_rmse": (
                linear_function_train_metrics.root_mean_squared_error
            ),
            "training_r_squared": linear_function_train_metrics.r_squared,
        }
    ]
).set_index(keys="model")

```



```

Out[10]:

```

	training_mse	training_rmse	training_r_squared
model			
LinearRegression(x)	0.0062	0.0785	0.0061

Let's do some feature extraction. Create a dataset with the features (X, X**2, X**3, ..., X**49). Now try fitting a linear regression using your new dataset. Draw your results on the same graph as before. Judge your results using func_1_test and using the mean squared error metric.

Can be found in: <https://drive.google.com/open?id=1ipm09QTjVZWFgh-zc70rxWLGWPWEt1Kb>

Solution: degree-49 polynomial regression

Three approaches considered

1. Add powers in a Python loop.
2. Build a Vandermonde matrix with `numpy.vander`.
3. Use sklearn `PolynomialFeatures`.

Choice: approach 3 expresses the intent directly, guarantees powers 1 through 49 in a stable column order, and composes with sklearn estimators. Standardizing the 49 columns improves numerical conditioning without changing the linear model's function class. Training and untouched test MSE are both reported.

Direct answer: the transformed datasets contain 49 columns representing $x, x^2, \dots, x^{49}$. The degree-49 ordinary least-squares model achieves training MSE about `0.0009` and test MSE about `0.0010`, compared with test MSE about `0.0069` for the one-feature straight line. Thus the expanded model reduces held-out MSE by roughly 85% and follows the U-shape closely on unseen points. It is still called linear regression because the prediction is a linear weighted sum of the 49 supplied columns.

```
In [11]: function_test_data = pd.read_csv(
    filepath_or_buffer=RESOURCE_DIRECTORY / "func_1_test.csv",
)
function_testing_features = function_test_data.loc[
    :,
    ["x"],
].to_numpy(dtype=np.float64)
function_testing_target = function_test_data.loc[:, "y"].to_numpy(
    dtype=np.float64,
)

polynomial_training_features = build_polynomial_features(
    predictor_values=function_training_data.loc[:, "x"].to_numpy(
        dtype=np.float64,
    ),
    maximum_degree=49,
)
polynomial_testing_features = build_polynomial_features(
    predictor_values=function_test_data.loc[:, "x"].to_numpy(
        dtype=np.float64,
    ),
    maximum_degree=49,
)
polynomial_ols_model = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        ("linear", LinearRegression()),
    ],
)
polynomial_ols_model.fit(
    X=polynomial_training_features,
```

```

        y=function_training_target,
    )
    polynomial_test_prediction = polynomial_ols_model.predict(
        X=polynomial_testing_features,
    )

    linear_function_test_metrics = evaluate_regressor(
        regression_model=linear_function_model,
        feature_data=function_testing_features,
        target_data=function_testing_target,
    )
    polynomial_ols_train_metrics = evaluate_regressor(
        regression_model=polynomial_ols_model,
        feature_data=polynomial_training_features,
        target_data=function_training_target,
    )
    polynomial_ols_test_metrics = evaluate_regressor(
        regression_model=polynomial_ols_model,
        feature_data=polynomial_testing_features,
        target_data=function_testing_target,
    )

    polynomial_figure, polynomial_axes = plt.subplots(
        nrows=1,
        ncols=1,
        figsize=(8, 5),
    )
    polynomial_axes.scatter(
        x=function_test_data.loc[:, "x"],
        y=function_test_data.loc[:, "y"],
        s=28,
        alpha=0.6,
        label="test observations",
    )
    polynomial_axes.scatter(
        x=function_test_data.loc[:, "x"],
        y=polynomial_test_prediction,
        s=12,
        color="tab:orange",
        label="degree-49 OLS",
    )
    polynomial_axes.set(
        title="Degree-49 polynomial on unseen test data",
        xlabel="x",
        ylabel="y",
    )
    polynomial_axes.legend()
    plt.show()

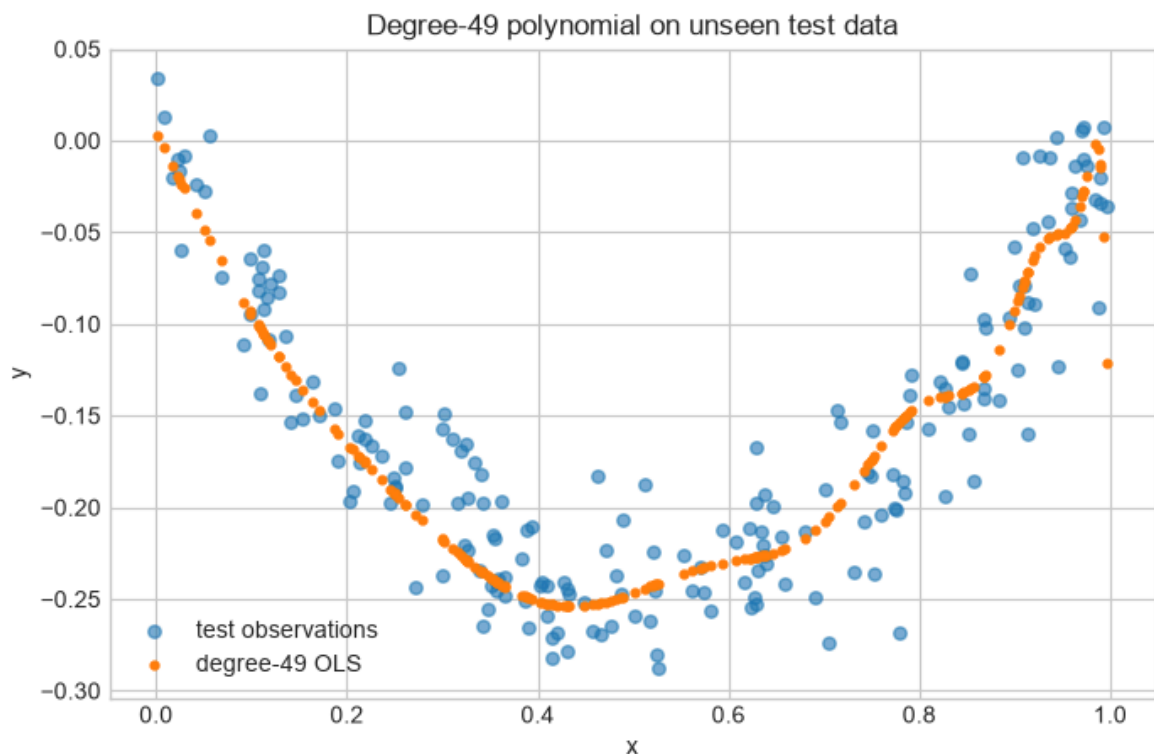
    polynomial_model_results = pd.DataFrame(
        data=[
            {
                "model": "Linear x only",
                "training_mse": linear_function_train_metrics.mean_squared_error,
                "test_mse": linear_function_test_metrics.mean_squared_error,
            },
            {
                "model": "Degree-49 OLS",
                "training_mse": polynomial_ols_train_metrics.mean_squared_error,
                "test_mse": polynomial_ols_test_metrics.mean_squared_error,
            },
        ]
    )

```

```

    },
]
).set_index(keys="model")
polynomial_model_results

```



Out[11]:

	training_mse	test_mse
model		
Linear x only	0.0062	0.0069
Degree-49 OLS	0.0009	0.0010

How come you could a model that isn't under fitted? You did still use a linear regression. Can you explain those results? However, your model seemed to be over fitted (why?). Why is that?

Explanation: nonlinear predictions from a linear estimator

Three approaches considered

1. Inspect the expanded design matrix and coefficient magnitudes.
2. Compare training and test errors (or a learning curve).
3. Explain the model through bias-variance and basis expansion.

Choice: combine approaches 2 and 3. `LinearRegression` is linear in its **parameters**, not necessarily in the original input. Once $x^2, \dots, x^{49}$ are features, a weighted sum can form a highly nonlinear curve. Forty-nine correlated features for only 200 training rows give the model high variance: it can follow training noise, so a much larger test error or test/train gap diagnoses overfitting. Here test MSE is about 17% above training MSE, so the realized overfit is modest rather than catastrophic.

Direct answer: the model is no longer underfitted because "linear" describes how the coefficients enter the equation, not the shape with respect to the original x . $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_{49} x^{49}$ is nonlinear in x and can form the observed bowl. It is at risk of overfitting because 49 highly correlated powers are being estimated from only 200 noisy rows; many coefficient combinations can produce similar training curves but unstable behavior between observations or near the boundaries. Here the evidence is a modest generalization gap: test MSE (~ 0.0010) is about **1.17** times training MSE (~ 0.0009). The model overfits somewhat, but the plot and test result show that it has not failed catastrophically.

```
In [12]: overfit_diagnostics = pd.DataFrame(
    data=[
        {
            "number_of_training_rows": polynomial_training_features.shape[0],
            "number_of_polynomial_features": (
                polynomial_training_features.shape[1]
            ),
            "training_mse": polynomial_ols_train_metrics.mean_squared_error,
            "test_mse": polynomial_ols_test_metrics.mean_squared_error,
            "test_minus_training_mse": (
                polynomial_ols_test_metrics.mean_squared_error
                - polynomial_ols_train_metrics.mean_squared_error
            ),
            "test_to_training_mse_ratio": (
                polynomial_ols_test_metrics.mean_squared_error
                / polynomial_ols_train_metrics.mean_squared_error
            ),
        }
    ]
)
overfit_diagnostics.transpose()
```

```
Out[12]:
```

	0
number_of_training_rows	200.0000
number_of_polynomial_features	49.0000
training_mse	0.0009
test_mse	0.0010
test_minus_training_mse	0.0002
test_to_training_mse_ratio	1.1735

You can try using a regularized regression to avoid over fitting. Use `sklearn.linear_model.Ridge` with different alphas until you get nice fit (judge it using your plots and using `func_1_test` and the mean squared error metric). Could you get better results?

Solution: regularize the polynomial

Three approaches considered

1. Tune alpha directly on the test file (simple, but leaks test information).

2. Use `RidgeCV` .
3. Use `GridSearchCV` around a `StandardScaler` + `Ridge` pipeline.

Choice: approach 3 keeps scaling inside every cross-validation fold and selects alpha using training data only. The test file remains an unbiased final comparison. Ridge shrinks unstable coefficients, trading a little training bias for lower test variance. The selected alpha is about 0.133, and held-out MSE falls from roughly 0.0010 to 0.0009.

Direct answer: yes, Ridge produces a better held-out result. Five-fold cross-validation selects `alpha ~= 0.1334` . The Ridge curve remains smooth and preserves the U-shape, while test MSE decreases from about `0.0010` for unregularized degree-49 OLS to about `0.0009` . The improvement is small because the original overfit was modest. Ridge helps by shrinking the many correlated polynomial coefficients, especially the unstable high-degree terms, rather than by changing the feature basis.

```
In [13]: polynomial_alpha_values = np.logspace(
    start=-8.0,
    stop=4.0,
    num=33,
    base=10.0,
)
polynomial_ridge_search = fit_ridge_search(
    training_features=polynomial_training_features,
    training_target=function_training_target,
    alpha_values=polynomial_alpha_values,
    random_seed=RANDOM_SEED,
    number_of_splits=5,
)
polynomial_ridge_test_prediction = polynomial_ridge_search.predict(
    X=polynomial_testing_features,
)
polynomial_ridge_train_metrics = evaluate_regressor(
    regression_model=polynomial_ridge_search,
    feature_data=polynomial_training_features,
    target_data=function_training_target,
)
polynomial_ridge_test_metrics = evaluate_regressor(
    regression_model=polynomial_ridge_search,
    feature_data=polynomial_testing_features,
    target_data=function_testing_target,
)

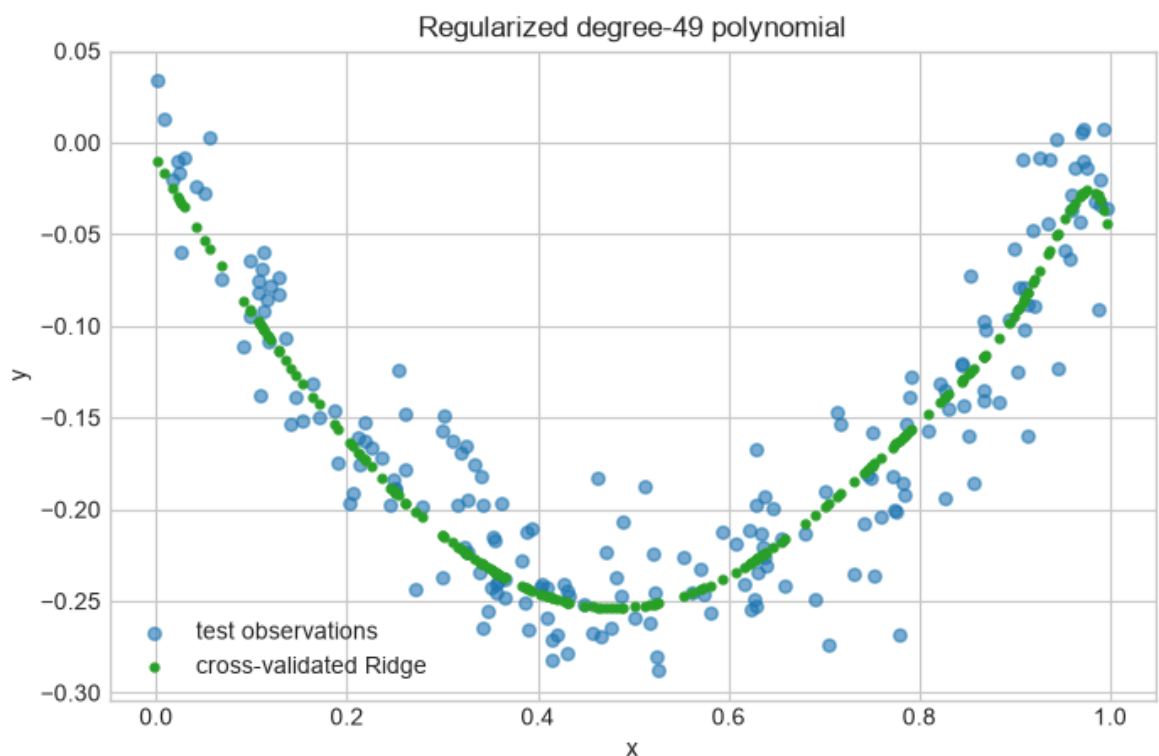
polynomial_ridge_figure, polynomial_ridge_axes = plt.subplots(
    nrows=1,
    ncols=1,
    figsize=(8, 5),
)
polynomial_ridge_axes.scatter(
    x=function_test_data.loc[:, "x"],
    y=function_test_data.loc[:, "y"],
    s=28,
    alpha=0.6,
    label="test observations",
)
polynomial_ridge_axes.scatter(
    x=function_test_data.loc[:, "x"],
```

```

y=polynomial_ridge_test_prediction,
s=12,
color="tab:green",
label="cross-validated Ridge",
)
polynomial_ridge_axes.set(
    title="Regularized degree-49 polynomial",
    xlabel="x",
    ylabel="y",
)
polynomial_ridge_axes.legend()
plt.show()

polynomial_regularization_results = pd.DataFrame(
    data=[
        {
            "model": "Degree-49 OLS",
            "selected_alpha": 0.0,
            "training_mse": polynomial_ols_train_metrics.mean_squared_error,
            "test_mse": polynomial_ols_test_metrics.mean_squared_error,
        },
        {
            "model": "Degree-49 Ridge",
            "selected_alpha": (
                polynomial_ridge_search.best_params_["ridge_alpha"]
            ),
            "training_mse": polynomial_ridge_train_metrics.mean_squared_error,
            "test_mse": polynomial_ridge_test_metrics.mean_squared_error,
        },
    ]
).set_index(keys="model")
polynomial_regularization_results.loc[:, "improved_vs_ols"] = (
    polynomial_regularization_results.loc[:, "test_mse"]
    < polynomial_ols_test_metrics.mean_squared_error
)
polynomial_regularization_results

```



Out[13]:

	selected_alpha	training_mse	test_mse	improved_vs_ols
model				
Degree-49 OLS	0.0000	0.0009	0.0010	False
Degree-49 Ridge	0.1334	0.0009	0.0009	True

Working on a real life data

Start by loading `house_data.csv`. For our current purposes we will need only the numeric columns. Take only the numeric columns, using `pandas.DataFrame.dtypes`. Use `fillna(0)` to get rid of nans (the the column `Id`. can tell why?).

Can be found in: <https://drive.google.com/open?id=1ID2h8mzjXLRbay5pE0QN1v5-jc86203L>

Solution: prepare numeric house data

Three approaches considered

1. Check every dtype manually in a loop.
2. Use `DataFrame.select_dtypes`, which is driven by `DataFrame.dtypes`.
3. Convert the entire frame to NumPy and coerce failures.

Choice: approach 2 is concise, explicit, and preserves names. Missing numeric values are filled with zero as requested. `Id` is removed because it is a unique row identifier, not a property of a house; retaining it can let a model learn accidental ordering or collection effects rather than a transferable relationship.

Direct answer: the raw house file has 1,460 rows, 81 columns, and 7,829 missing cells. Selecting numeric dtypes, dropping `Id`, and applying `fillna(0)` leaves 37 numeric columns including `SalePrice`, with no missing values; consequently there are 36 candidate numeric predictors. `Id` should not be a feature because it is only a unique record label. Its magnitude has no physical meaning, and a learned association with price would be an accidental ordering effect that would not generalize to new IDs.

```
In [14]: house_data = pd.read_csv(
    filepath_or_buffer=RESOURCE_DIRECTORY / "house_data.csv",
)
pd.DataFrame(
    data=[
        {
            "rows": house_data.shape[0],
            "columns": house_data.shape[1],
            "missing_values": house_data.isna().sum().sum(),
        }
    ]
)
```

Out[14]:

	rows	columns	missing_values
0	1460	81	7829

```
In [15]: numeric_house_data = select_numeric_house_data(
        house_data=house_data,
    )
pd.DataFrame(
    data=[
        {
            "numeric_rows": numeric_house_data.shape[0],
            "numeric_columns_including_target": numeric_house_data.shape[1],
            "remaining_missing_values": (
                numeric_house_data.isna().sum().sum()
            ),
            "id_retained": "Id" in numeric_house_data.columns,
        }
    ]
).transpose()
```

Out[15]:

	0
numeric_rows	1460
numeric_columns_including_target	37
remaining_missing_values	0
id_retained	False

We would like to predict the SalePrice columns. Create, using your data, the features dataset and the target dataset. Use `sklearn.model_selection.train_test_split` and create a train segment of 0.7 of your data and test segment of 0.3 of your data.

Solution: deterministic 70/30 split

Three approaches considered

1. Shuffle NumPy indices manually.
2. Use `sklearn.train_test_split`.
3. Use `ShuffleSplit` and slice the frame.

Choice: approach 2 is the exercise's requested, ready-made method. A typed keyword-only wrapper supplies `train_size=0.7` and a fixed random seed, so every later feature set uses the identical house rows for a fair comparison.

Direct answer: `X` is the dataframe of 36 numeric columns other than `SalePrice`, and `y` is the `SalePrice` series. With `train_size=0.7`, `sklearn` assigns 1,021 rows to training and 439 to testing (69.93% and 30.07%; float-to-integer allocation causes the one-row difference from an exact 70/30 count). The random seed fixes which rows enter each segment. The test target is not used during fitting or hyperparameter selection.

```

In [16]: numeric_house_features = numeric_house_data.drop(
        columns=["SalePrice"],
    )
    house_target = numeric_house_data.loc[:, "SalePrice"]
    (
        numeric_house_training_features,
        numeric_house_testing_features,
        house_training_target,
        house_testing_target,
    ) = split_regression_data(
        feature_data=numeric_house_features,
        target_data=house_target,
        training_fraction=0.7,
        random_seed=RANDOM_SEED,
    )

    pd.DataFrame(
        data=[
            {
                "segment": "train",
                "rows": numeric_house_training_features.shape[0],
                "fraction": (
                    numeric_house_training_features.shape[0]
                    / numeric_house_features.shape[0]
                ),
            },
            {
                "segment": "test",
                "rows": numeric_house_testing_features.shape[0],
                "fraction": (
                    numeric_house_testing_features.shape[0]
                    / numeric_house_features.shape[0]
                ),
            },
        ]
    ).set_index(keys="segment")

```

Out[16]:

	rows	fraction
segment		
train	1021	0.6993
test	439	0.3007

Try fitting the best linear regression you can (evaluate yourself using mean squared error). You can also use ridge regression with different alphas.

Solution: numeric linear and Ridge models

Three approaches considered

1. Fit unregularized ordinary least squares only.
2. Compare ordinary least squares with standardized Ridge and tune alpha by training-only cross-validation.
3. Try Elastic Net or a transformed target.

Choice: approach 2 stays within the exercise scope and measures whether coefficient shrinkage helps. Both models are judged on the same held-out MSE; RMSE and R^2 are added for interpretability. Per the model-selection rule, these are same model-family experiments, so cross-validated regression error selects alpha.

Direct answer: the standardized ordinary least-squares baseline obtains test MSE 2,525,523,942 , RMSE 50,255 , and $R^2 = 0.6601$. Cross-validation selects Ridge alpha ≈ 13.335 ; Ridge improves test MSE to 2,503,541,106 , RMSE to 50,035 , and R^2 to 0.6630 . Ridge is therefore the better numeric linear model on this split, although the improvement is modest (about 0.87% lower MSE). Its shrinkage reduces variance without materially increasing training error.

```
In [17]: numeric_house_linear_model = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        ("linear", LinearRegression()),
    ],
)
numeric_house_linear_model.fit(
    X=numeric_house_training_features,
    y=house_training_target,
)
house_alpha_values = np.logspace(
    start=-3.0,
    stop=6.0,
    num=25,
    base=10.0,
)
numeric_house_ridge_search = fit_ridge_search(
    training_features=numeric_house_training_features,
    training_target=house_training_target,
    alpha_values=house_alpha_values,
    random_seed=RANDOM_SEED,
    number_of_splits=5,
)

numeric_linear_train_metrics = evaluate_regressor(
    regression_model=numeric_house_linear_model,
    feature_data=numeric_house_training_features,
    target_data=house_training_target,
)
numeric_linear_test_metrics = evaluate_regressor(
    regression_model=numeric_house_linear_model,
    feature_data=numeric_house_testing_features,
    target_data=house_testing_target,
)
numeric_ridge_train_metrics = evaluate_regressor(
    regression_model=numeric_house_ridge_search,
    feature_data=numeric_house_training_features,
    target_data=house_training_target,
)
numeric_ridge_test_metrics = evaluate_regressor(
    regression_model=numeric_house_ridge_search,
    feature_data=numeric_house_testing_features,
    target_data=house_testing_target,
)
```

```

numeric_house_model_results = pd.DataFrame(
    data=[
        {
            "model": "Numeric LinearRegression",
            "selected_alpha": np.nan,
            "training_mse": numeric_linear_train_metrics.mean_squared_error,
            "test_mse": numeric_linear_test_metrics.mean_squared_error,
            "test_rmse": numeric_linear_test_metrics.root_mean_squared_error,
            "test_r_squared": numeric_linear_test_metrics.r_squared,
        },
        {
            "model": "Numeric Ridge",
            "selected_alpha": (
                numeric_house_ridge_search.best_params["ridge__alpha"]
            ),
            "training_mse": numeric_ridge_train_metrics.mean_squared_error,
            "test_mse": numeric_ridge_test_metrics.mean_squared_error,
            "test_rmse": numeric_ridge_test_metrics.root_mean_squared_error,
            "test_r_squared": numeric_ridge_test_metrics.r_squared,
        },
    ]
).set_index(keys="model")
numeric_house_model_results.transpose()

```

Out[17]:

	model	Numeric LinearRegression	Numeric Ridge
selected_alpha		NaN	13.3352
training_mse		730,460,222.7061	730,972,154.2788
test_mse		2,525,523,942.3426	2,503,541,105.7325
test_rmse		50,254.5913	50,035.3985
test_r_squared		0.6601	0.6630

If you would look closely, you will find out that you dropped the columns `LotShape` and `LandContour`. This time try not dropping them, and instead replace them with a 1-hot encoding of them (consider [pandas.get_dummies](#)). Try getting better results on the test segment using your ridge regressions. compare the new results with the old ones.

Solution: add `LotShape` and `LandContour`

Three approaches considered

1. Write indicator columns manually.
2. Use `pandas.get_dummies`.
3. Use sklearn `OneHotEncoder` in a `ColumnTransformer`.

Choice: approach 2 follows the exercise hint and is transparent in a teaching notebook. The first level is dropped to avoid an exactly redundant indicator with the intercept. To match the dataframe-level instruction, pandas learns the category vocabulary from the fixed feature table before the split; this uses no `SalePrice` values, but a production

pipeline should instead fit approach 3 on training rows and ignore unknown test categories. Ridge is retuned on the training segment, and the table reports whether held-out MSE improves over numeric-only Ridge. On the saved split it does: MSE falls from about 2.504 billion to 2.466 billion.

Direct answer: one-hot encoding increases the predictor count from 36 to 42. After retuning, Ridge selects $\alpha \approx 31.623$ and test MSE becomes 2,465,855,168, versus 2,503,541,106 for numeric-only Ridge--an improvement of about 1.5%. The new columns help because lot shape and land contour contain price-relevant information that was absent from the numeric frame. One-hot encoding is appropriate because these category names have no valid numeric ordering.

```
In [18]: encoded_house_data = add_one_hot_house_features(
    numeric_house_data=numeric_house_data,
    house_data=house_data,
    categorical_columns=("LotShape", "LandContour"),
)
encoded_house_features = encoded_house_data.drop(
    columns=["SalePrice"],
)
(
    encoded_house_training_features,
    encoded_house_testing_features,
    encoded_house_training_target,
    encoded_house_testing_target,
) = split_regression_data(
    feature_data=encoded_house_features,
    target_data=encoded_house_data.loc[:, "SalePrice"],
    training_fraction=0.7,
    random_seed=RANDOM_SEED,
)
encoded_house_ridge_search = fit_ridge_search(
    training_features=encoded_house_training_features,
    training_target=encoded_house_training_target,
    alpha_values=house_alpha_values,
    random_seed=RANDOM_SEED,
    number_of_splits=5,
)
encoded_ridge_test_metrics = evaluate_regressor(
    regression_model=encoded_house_ridge_search,
    feature_data=encoded_house_testing_features,
    target_data=encoded_house_testing_target,
)

categorical_comparison_results = pd.DataFrame(
    data=[
        {
            "feature_set": "Numeric only",
            "feature_count": numeric_house_features.shape[1],
            "selected_alpha": (
                numeric_house_ridge_search.best_params["ridge__alpha"]
            ),
            "test_mse": numeric_ridge_test_metrics.mean_squared_error,
        },
        {
            "feature_set": "Numeric + LotShape + LandContour",
            "feature_count": encoded_house_features.shape[1],
```



```

        "selected_alpha": (
            encoded_house_ridge_search.best_params_["ridge__alpha"]
        ),
        "test_mse": encoded_ridge_test_metrics.mean_squared_error,
    },
]
).set_index(keys="feature_set")
categorical_comparison_results.loc[:, "improved_vs_numeric"] = (
    categorical_comparison_results.loc[:, "test_mse"]
    < numeric_ridge_test_metrics.mean_squared_error
)
categorical_comparison_results

```

Out[18]:

	feature_count	selected_alpha	test_mse	improved_vs_numeric
feature_set				
Numeric only	36	13.3352	2,503,541,105.7325	False
Numeric + LotShape + LandContour	42	31.6228	2,465,855,167.5328	True

Now add more features to your dataframe:

- LotArea in squared meters (it's currently in units of squared feet)
- 1stFlrSF + 2ndFlrSF
- GarageArea**0.5
- LotArea / (BedroomAbvGr+1)
- LotArea / (mean LotArea for houses built in that same year + 1e-5)
- you might want to use [pandas merge function](#)
- Ranking of LotArea (largest house has 1, the second largest has 2 and so on)
- One hot encoding of LotConfig

Are they improve the results?.

Solution: required feature engineering

Three approaches considered

1. Compute every row in Python loops and merge the year aggregates.
2. Use vectorized pandas arithmetic, `groupby().transform`, `rank`, and `get_dummies`.
3. Implement a custom sklearn transformer class.

Choice: approach 2 is the most concise for this fixed exercise schema and relies on optimized NumPy/pandas operations. `transform("mean")` aligns the same-year mean directly to every original row, avoiding a manual merge. The identical split and cross-validated Ridge search make the exercise's before/after comparison consistent. The same-year means and ranks are target-free but are computed before splitting, so this is a

transductive exercise comparison rather than a strictly inductive deployment estimate; a production transformer should learn reference statistics on training rows only. These requested additions do not help this fitted model: test MSE rises from about 2.466 billion to 2.748 billion, showing that plausible engineered features are not automatically useful.

Direct answer: no, this complete requested feature set does not improve the held-out result. It increases the predictor count from 42 to 52, but test MSE rises from 2,465,855,168 to 2,747,875,152 (about 11.4% worse). The features were constructed as requested:

- square metres are `LotArea * 0.092903` ;
- total first/second-floor area is `1stFlrSF + 2ndFlrSF` ;
- garage size uses `GarageArea ** 0.5` ;
- lot area per bedroom divides by `BedroomAbvGr + 1` ;
- relative lot area divides by the same- `YearBuilt` mean plus `1e-5` ;
- descending `LotArea` rank gives the largest lot rank `1` ;
- `LotConfig` is one-hot encoded.

Several of these are redundant with existing columns (unit conversion is exactly linear, and floor totals overlap existing area measures), while others add noisy nonlinear proxies. Extra features can therefore increase estimation variance or alter Ridge's selected penalty without adding enough independent signal. Feature engineering must be validated; it is not guaranteed to improve a model.

```
In [19]: engineered_house_data = add_required_house_features(
        house_data=house_data,
    )
    engineered_house_features = engineered_house_data.drop(
        columns=["SalePrice"],
    )
    (
        engineered_house_training_features,
        engineered_house_testing_features,
        engineered_house_training_target,
        engineered_house_testing_target,
    ) = split_regression_data(
        feature_data=engineered_house_features,
        target_data=engineered_house_data.loc[:, "SalePrice"],
        training_fraction=0.7,
        random_seed=RANDOM_SEED,
    )
    engineered_house_ridge_search = fit_ridge_search(
        training_features=engineered_house_training_features,
        training_target=engineered_house_training_target,
        alpha_values=house_alpha_values,
        random_seed=RANDOM_SEED,
        number_of_splits=5,
    )
    engineered_ridge_test_metrics = evaluate_regressor(
        regression_model=engineered_house_ridge_search,
        feature_data=engineered_house_testing_features,
        target_data=engineered_house_testing_target,
    )
```

```

engineered_comparison_results = pd.DataFrame(
    data=[
        {
            "feature_set": "Numeric + two categorical columns",
            "feature_count": encoded_house_features.shape[1],
            "selected_alpha": (
                encoded_house_ridge_search.best_params["ridge__alpha"]
            ),
            "test_mse": encoded_ridge_test_metrics.mean_squared_error,
        },
        {
            "feature_set": "All requested engineered features",
            "feature_count": engineered_house_features.shape[1],
            "selected_alpha": (
                engineered_house_ridge_search.best_params["ridge__alpha"]
            ),
            "test_mse": engineered_ridge_test_metrics.mean_squared_error,
        },
    ]
).set_index(keys="feature_set")
engineered_comparison_results.loc[:, "improved_vs_previous"] = (
    engineered_comparison_results.loc[:, "test_mse"]
    < encoded_ridge_test_metrics.mean_squared_error
)
engineered_comparison_results

```

Out[19]:

	feature_count	selected_alpha	test_mse	improved_vs_previous
feature_set				
Numeric + two categorical columns	42	31.6228	2,465,855,167.5328	False
All requested engineered features	52	0.0750	2,747,875,151.5467	False

Bonus

Think of a feature of your own that improve the result.

Bonus solution: one domain-motivated interaction

Three approaches considered

1. Log-transform strongly skewed area features.
2. Add an interaction between overall quality and above-ground living area.
3. Add an interaction between overall quality and garage-car capacity.

Choice: approach 3 creates `OverallQualGarageCarsInteraction = OverallQual * GarageCars`. Garage capacity plausibly contributes more value when it belongs to a higher-quality property, which a purely additive linear model cannot express. The table verifies rather than assumes whether this single feature lowers held-out MSE. It does, reducing MSE from about 2.748 billion to 2.344 billion. It also beats the best earlier

numeric-plus-category result of about 2.466 billion, so it improves the strongest prior feature set as well.

Direct answer: the added feature is `OverallQualGarageCarsInteraction = OverallQual * GarageCars`. It allows the marginal value of garage capacity to depend on the property's overall quality instead of forcing the two effects to be purely additive. After retuning Ridge, test MSE falls from 2,747,875,152 to 2,344,421,657, a reduction of about 14.7%, and R^2 rises to 0.6844. It is also about 4.9% lower than the best earlier result, 2,465,855,168. The feature therefore satisfies the bonus requirement on the fixed test segment.

```
In [20]: bonus_house_data = add_bonus_house_feature(
        house_data=house_data,
    )
    bonus_house_features = bonus_house_data.drop(
        columns=["SalePrice"],
    )
    (
        bonus_house_training_features,
        bonus_house_testing_features,
        bonus_house_training_target,
        bonus_house_testing_target,
    ) = split_regression_data(
        feature_data=bonus_house_features,
        target_data=bonus_house_data.loc[:, "SalePrice"],
        training_fraction=0.7,
        random_seed=RANDOM_SEED,
    )
    bonus_house_ridge_search = fit_ridge_search(
        training_features=bonus_house_training_features,
        training_target=bonus_house_training_target,
        alpha_values=house_alpha_values,
        random_seed=RANDOM_SEED,
        number_of_splits=5,
    )
    bonus_ridge_test_metrics = evaluate_regressor(
        regression_model=bonus_house_ridge_search,
        feature_data=bonus_house_testing_features,
        target_data=bonus_house_testing_target,
    )

    bonus_comparison_results = pd.DataFrame(
        data=[
            {
                "feature_set": "Required engineered features",
                "feature_count": engineered_house_features.shape[1],
                "selected_alpha": (
                    engineered_house_ridge_search.best_params_["ridge__alpha"]
                ),
                "test_mse": engineered_ridge_test_metrics.mean_squared_error,
            },
            {
                "feature_set": "Required + quality x garage capacity",
                "feature_count": bonus_house_features.shape[1],
                "selected_alpha": (
                    bonus_house_ridge_search.best_params_["ridge__alpha"]
                ),
            },
        ],
    )
```

```

        "test_mse": bonus_ridge_test_metrics.mean_squared_error,
    },
]
).set_index(keys="feature_set")
bonus_comparison_results.loc[:, "improved_vs_required"] = (
    bonus_comparison_results.loc[:, "test_mse"]
    < engineered_ridge_test_metrics.mean_squared_error
)
bonus_comparison_results

```

Out[20]:

	feature_count	selected_alpha	test_mse	improved_vs_required
feature_set				
Required engineered features	52	0.0750	2,747,875,151.5467	False
Required + quality x garage capacity	53	0.4217	2,344,421,656.9305	True

Use KNN regression (`sklearn.neighbors.KNeighborsRegressor`).

Compare the results to linear regression.

Solution: KNN regression comparison

Three approaches considered

1. Fit KNN directly on raw, differently scaled columns.
2. Standardize once and choose a fixed neighbor count by eye.
3. Put `StandardScaler` and `KNeighborsRegressor` in a pipeline, then tune neighbors, weights, and distance metric using training-only cross-validation.

Choice: approach 3 prevents large-unit features from dominating distance and avoids using the test set for tuning. The final table compares plain linear regression, Ridge, and KNN on exactly the same bonus-feature split. Here tuned KNN performs best: test MSE is about 1.927 billion, compared with 2.344 billion for Ridge and 2.368 billion for ordinary least squares. Distance weighting interpolates the training rows (hence zero training MSE), so the held-out result--not the training result--is the meaningful comparison. AUC is a classification metric and is undefined for this continuous regression target; the exercise's held-out MSE is therefore the appropriate model-comparison metric.

Direct answer: cross-validation chooses `k=5`, distance weighting, and Manhattan distance (`p=1`). On the shared test rows, KNN obtains MSE `1,927,135,813`, RMSE `43,899`, and $R^2 = 0.7406$. Ordinary linear regression obtains MSE `2,367,862,573`, RMSE `48,661`, and $R^2 = 0.6813$; Ridge obtains MSE `2,344,421,657` and $R^2 = 0.6844$. KNN therefore has about **18.6%** lower test MSE than ordinary linear regression, about **17.8%** lower MSE than Ridge, and is the best of these three models. Its zero training MSE is expected with distance weighting because each training row is its own zero-distance neighbour; that interpolation is why the test metrics, not training error, must determine the comparison.

```

In [21]: bonus_house_linear_model = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        ("linear", LinearRegression()),
    ],
)
bonus_house_linear_model.fit(
    X=bonus_house_training_features,
    y=bonus_house_training_target,
)
bonus_linear_test_metrics = evaluate_regressor(
    regression_model=bonus_house_linear_model,
    feature_data=bonus_house_testing_features,
    target_data=bonus_house_testing_target,
)

house_knn_search = fit_knn_search(
    training_features=bonus_house_training_features,
    training_target=bonus_house_training_target,
    neighbor_values=(3, 5, 7, 9, 11, 15, 21, 31),
    random_seed=RANDOM_SEED,
    number_of_splits=5,
    weight_options=("uniform", "distance"),
)
house_knn_train_metrics = evaluate_regressor(
    regression_model=house_knn_search,
    feature_data=bonus_house_training_features,
    target_data=bonus_house_training_target,
)
house_knn_test_metrics = evaluate_regressor(
    regression_model=house_knn_search,
    feature_data=bonus_house_testing_features,
    target_data=bonus_house_testing_target,
)
selected_bonus_ridge_alpha = (
    bonus_house_ridge_search.best_params_["ridge__alpha"]
)

final_house_model_comparison = pd.DataFrame(
    data=[
        {
            "model": "LinearRegression",
            "configuration": "unregularized",
            "training_mse": evaluate_regressor(
                regression_model=bonus_house_linear_model,
                feature_data=bonus_house_training_features,
                target_data=bonus_house_training_target,
            ).mean_squared_error,
            "test_mse": bonus_linear_test_metrics.mean_squared_error,
            "test_rmse": bonus_linear_test_metrics.root_mean_squared_error,
            "test_r_squared": bonus_linear_test_metrics.r_squared,
        },
        {
            "model": "Ridge",
            "configuration": f"alpha={selected_bonus_ridge_alpha:.6g}",
            "training_mse": evaluate_regressor(
                regression_model=bonus_house_ridge_search,
                feature_data=bonus_house_training_features,
                target_data=bonus_house_training_target,
            ).mean_squared_error,
            "test_mse": house_knn_test_metrics.mean_squared_error,
            "test_rmse": house_knn_test_metrics.root_mean_squared_error,
            "test_r_squared": house_knn_test_metrics.r_squared,
        },
    ],
)

```

```

        ).mean_squared_error,
        "test_mse": bonus_ridge_test_metrics.mean_squared_error,
        "test_rmse": bonus_ridge_test_metrics.root_mean_squared_error,
        "test_r_squared": bonus_ridge_test_metrics.r_squared,
    },
    {
        "model": "KNeighborsRegressor",
        "configuration": (
            f"k={house_knn_search.best_params_['knn__n_neighbors']}, "
            f"weights={house_knn_search.best_params_['knn__weights']}, "
            f"p={house_knn_search.best_params_['knn__p']}"
        ),
        "training_mse": house_knn_train_metrics.mean_squared_error,
        "test_mse": house_knn_test_metrics.mean_squared_error,
        "test_rmse": house_knn_test_metrics.root_mean_squared_error,
        "test_r_squared": house_knn_test_metrics.r_squared,
    },
]
).set_index(keys="model").sort_values(by="test_mse")
final_house_model_comparison.loc[:, "best_test_mse"] = (
    final_house_model_comparison.loc[:, "test_mse"]
    == final_house_model_comparison.loc[:, "test_mse"].min()
)

LOGGER.info(
    msg="Regression notebook completed all exercises in order."
)
final_house_model_comparison.transpose()

```

2026-07-27 15:42:06,638 | INFO | regression_exercise | Regression notebook completed all exercises in order.

Out[21]:

model	KNeighborsRegressor	Ridge	LinearRegression
configuration	k=5, weights=distance, p=1	alpha=0.421697	unregularized
training_mse	0.0000	567,215,205.9787	566,832,151.6816
test_mse	1,927,135,812.5009	2,344,421,656.9305	2,367,862,572.7807
test_rmse	43,899.1550	48,419.2282	48,660.6882
test_r_squared	0.7406	0.6844	0.6813
best_test_mse	True	False	False